Parallel Loops

Victor Eijkhout

2026 PCSE

Loop parallelism

Much of parallelism in scientific computing is in loops:

- Vector updates and inner products
- Matrix-vector and matrix-matrix operations
- Finite Element meshes
- Multigrid

Work distribution

- Suppose loop iterations are independent:
- Distribute them over the threads:
- Use *omp_get_thread_num* to determine disjoint subsets.

```
1 #pragma omp parallel
2 {
3   int threadnum = omp_get_thread_num(),
4     numthreads = omp_get_num_threads();
5   int low = N*threadnum/numthreads,
6     high = N*(threadnum+1)/numthreads;
7   for (int i=low; i<high; i++)
8     // do something with i
9 }
```

Discuss.

Loops are easy

for directive before a loop:

```
1 #pragma omp parallel
2 {
3 #pragma omp for
4   for ( int i=0; i<N; i++)
5     ... something with i ...
6 }
```

- OpenMP does the partitioning of the iteration space
- blocks by default but other 'schedules' possible
- even dynamic load balancing.

Workshare constructs

Here's the two-step parallelization in OpenMP:

- You use the `parallel` directive to create a team of threads;
- then you use a 'workshare' construct to distribute the work over the team.
- For loops that is the `for` (or `do`) construct.

Workshare construct for loops

Parallel and workshare together

```
1 #pragma omp parallel for
2   for (i=0; i<N; i++)
3     ... something with i ...
```

Range syntax

Parallel loops in C++ can use range-based syntax as of OpenMP-5.0:

```
1  // vecdata.cpp
2  vector<float> values(N);
3  double tstart = omp_get_wtime();
4
5  #pragma omp parallel for
6  for ( auto& elt : values ) {
7    elt = 5.f;
8  }
9
10 float sum{0.f};
11 #pragma omp parallel for reduction(+:sum)
12 for ( auto elt : values ) {
13   sum += elt;
14 }
```

Tests show exactly the same speedup as the C code.

Ranges library

The C++20 *ranges* library is supported:

```
#pragma omp parallel for reduction(+:itcount)
for ( auto e : data
              | std::ranges::views::drop(1) )
        itcount += e;
#pragma omp parallel for reduction(+:itcount)
for ( auto e : data
              | std::ranges::views::transform
              ( [](auto e ) { return 2*e; } ) )
        itcount += e;
```

Index ranges

Use *iota_view* to obtain indices:

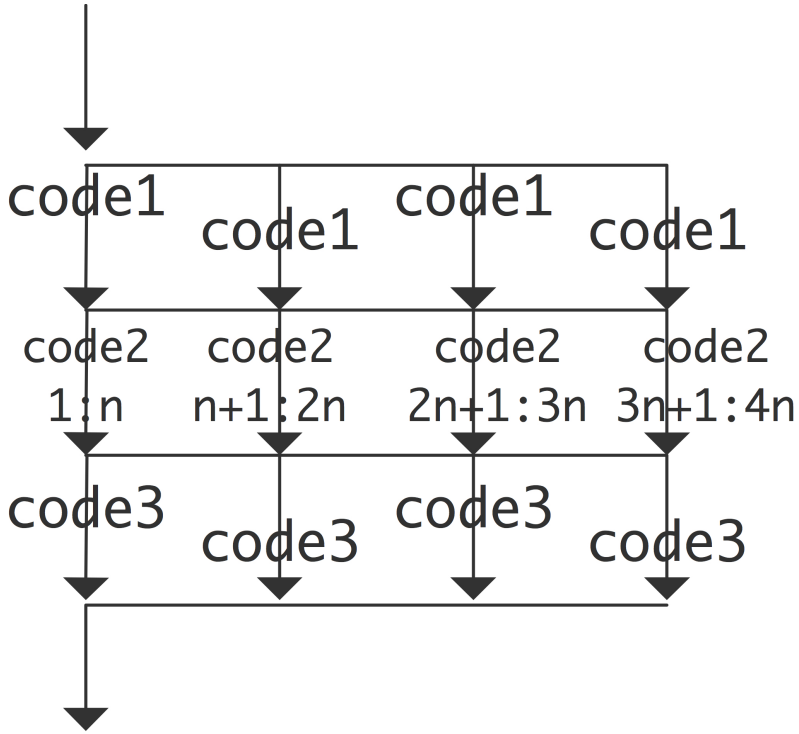
```
1 // iota.cpp
2 vector<long> data(N);
3 # pragma omp parallel for
4 for ( auto i : std::ranges::iota_view( 0UZ,data.size() ) )
5   data[i] = f(i);
```

Note that this uses C++23 suffix for *unsigned size_t*. For older versions:

```
1 iota_view( static_cast<size_t>(0),data.size() )
```

Stuff inside a parallel region

```
1 #pragma omp parallel
2 {
3   code1();
4   #pragma omp for
5   for (int i=1; i<=4*N; i++) {
6     code2();
7   }
8   code3();
9 }
```



Loops are static

- No `break` or so.
- `continue` or `cycle` allowed.
- Fixed upper bound;
- Simple loop increment;
- $\Rightarrow$ OpenMP needs to be able to calculate the number of iterations in advance.
- No `while` loops.

When is a loop parallel?

It's up to you!

This is parallelizable:

```
1   for (int i=low; i<hi; i++)
2      x[i] = // expression without x
```

Is this?

```
1 for (int i=low; i<hi; i++)
2    x[ f(i) ] = // expression
```

Histograms / binning.

Exercise 1

Consider the code

```
1 for (int i=0; i<n; i++)
2   x[i/2] += f(i)
```

Argue that this does not satisfy the above consition. Can you rewrite this loop to be parallelizable?

Exercise 2 `vectorsum`

Run the following snippet

```
1 // vectorsum.c
2 for ( int iloop=0; iloop<nloops; ++iloop ) {
3   for ( int i=0; i<vectorsize; ++i ) {
4     outvec[i] += invec[i]*loopcoeff[iloop];
5   }
6 }
```

1. Sequentially
2. On one thread
3. With more than one thread.

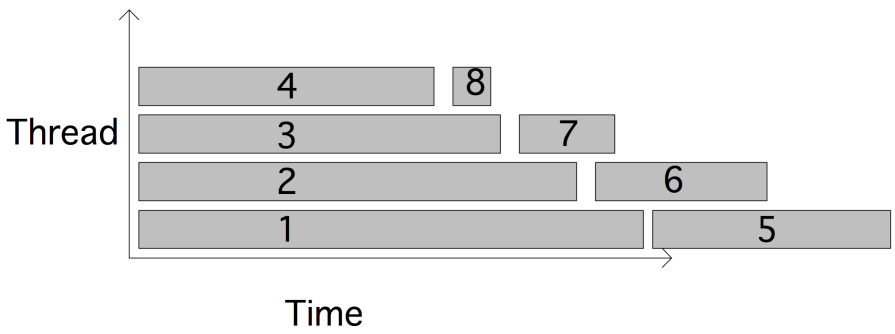
Do a scaling study. What all can you do to improve performance? Read back through previous slides.

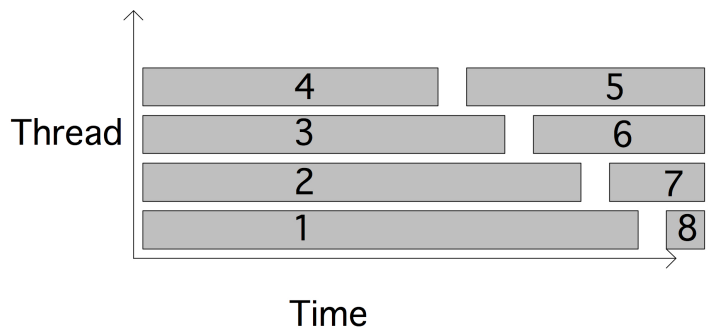
More about loops

Loop schedules

- Static scheduling of iterations.
  (default in practice though not formally)
  Very efficient. Good if all iterations take the same amount of time.
  `schedule(static)`
- Other possibility: dynamic.
  Runtime overhead; better if iterations do not take the same amount of time.
  `schedule(dynamic)`

Four threads, 8 tasks of decreasing size
dynamic schedule is better:

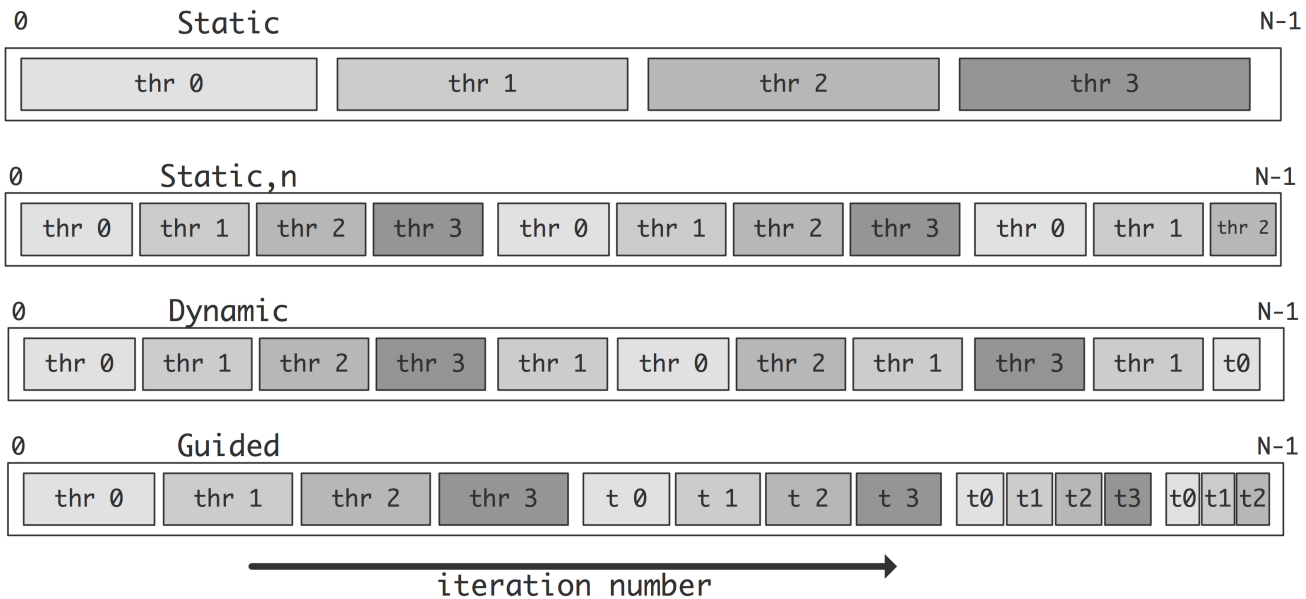




Chunk size

With N iterations and t threads:

- Static: each thread gets N/t iterations.
  explicit chunk size: `schedule(static,123)`
- Dynamic: each thread gets 1 iteration at a time
  explicit chunk size: `schedule(dynamic,45)`



Guided schedule

Use decreasing chunk size (with optional minimum chunk):
`schedule(guided,6)`

More flexible than dynamic, less overhead than dynamic.

Collapse

- Parallelize loop nest
- More iterations ⇒ better performance
- Inner loop needs to be directly nested
  bounds simple function of outer bounds and var

```
1  // triangle.c
2  #pragma omp parallel for collapse(2)
3    for ( int i=0; i<vectorsize; ++i ) {
4      for ( int j=i+1; j<vectorsize; ++j ) {
5        double
6          dx= particles[XCOORD(i)]-particles[XCOORD(j)],
7          dy= particles[YCOORD(i)]-particles[YCOORD(j)];
8        double r = sqrt( dx*dx + dy*dy ),
9          f = (particles[MASS(i)]*particles[MASS(j)])/(r*r);
10     }
11   }
```

Ordered loops

- Ordered iterations: normally OpenMP can execute iterations in any sequence. You can force ordering if you absolutely have to.
- Bad for performance and generally not needed.
- Use for I/O:

```
1 #pragma omp parallel for ordered schedule(dynamic)
2 for ( i=... ) {
3   x[i] = ...
4   #pragma omp ordered
5     printf(x[i])
6 }
```

Loop data

Private vs shared

Statements such as:

```
1 #pragma omp parallel private(x)
2 #pragma omp parallel shared(y)
3 #pragma omp parallel default(none) shared(x) private(y)
```

- Shared data: comes from outside the parallel region
- Private data: either private versions of outside, or local defined
- Default is shared. Dangerous!
- Parallel loop variables are always private.

Where is the bug?

```
1   int i,j;
2   double temp;
3 #pragma omp parallel for private(temp)
4    for(i=0;i<N;i++){
5     for (j=0;j<M;j++){
6        temp = b[i]*c[j];
7        a[i][j] = temp * temp + d[i];
8 } }
```

Is there a different way of handling `temp`?

More private/shared

```
1  float x=5.5;
2  #pragma omp parallel firstprivate(x)
3    // x is now private, and has value 5.5
4
5  float y;
6  #pragma omp parallel for lastprivate(y)
7  for ( /* looping */ ) {
8    // stuff
9    y = something;
10 }
11 // now y has the sequentially last value
```

Reduction

Exercise 3

Extend the program from exercise **??**. Make a complete program based on these lines:

Code:

```
1 // reduct.c
2 int tsum=0;
3 #pragma omp parallel
4 {
5   tsum += // expression
6 }
7 printf("Sum is %d\n",tsum);
```

Output:

```
1 With 4 threads, sum s/b
      ↪6
2 Sum is 6
3 Sum is 5
4 Sum is 1
5 Sum is 4
6 Sum is 6
7 Sum is 5
8 Sum is 6
9 Sum is 5
10 Sum is 3
11 Sum is 4
```

Compile and run again. (In fact, run your program a number of times.) Do you see something unexpected? Can you think of an explanation?

Shared memory problems

Race condition: simultaneous update of shared data:

process 1: `I=I+2`
process 2: `I=I+3`

Results can be indeterminate:

scenario 1.		scenario 2.		scenario 3.	
I = 0					
read I = 0	read I = 0	read I = 0	read I = 0	read I = 0	
compute I = 2	compute I = 3	compute I = 2	compute I = 3	compute I = 2	
write I = 2			write I = 3	write I = 2	
	write I = 3	write I = 2			read I = 2
					compute I = 5
					write I = 5
I = 3		I = 2		I = 5	

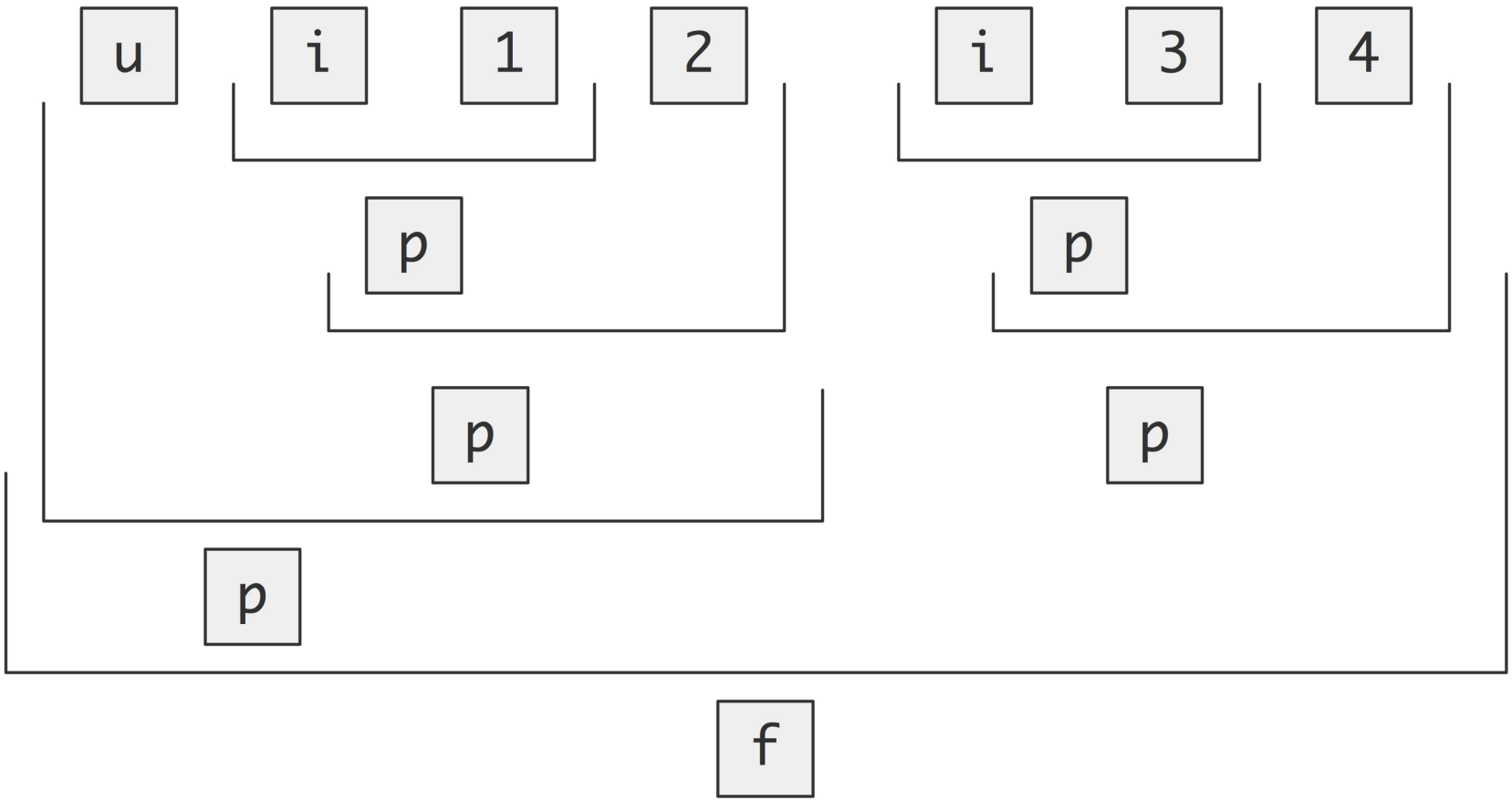
Reductions

- Inner product loop:

```
1 // reductsum.c
2 float s = 0;
3 #pragma omp parallel for reduction(+:s)
4 for (int idata=0; idata<ndata; idata++)
5   s += x[idata]*y[idata];
```

- Use the `reduction(+:s)` clause.
- All the usual operations are available; you can also make your own.

How OMP reducts



Exercise 4 `pi`

Compute π by *numerical integration*. We use the fact that π is the area of the unit circle, and we approximate this by computing the area of a quarter circle using *Riemann sums*.

- Let $f(x) = \sqrt{1-x^2}$ be the function that describes the quarter circle for $x = 0 \ldots 1$;
- Then we compute

$$\pi/4 \approx \sum_{i=0}^{N-1} \Delta x f(x_i) \qquad \text{where } x_i = i\Delta x \text{ and } \Delta x = 1/N$$

Write a program for this, and parallelize it using OpenMP parallel for directives. Measure attained speedup.

1. Put a *parallel* directive around your loop. Does it still compute the right result? Does the time go down with the number of threads? (The answers should be no and no.)
2. Change the parallel to parallel for (or parallel do). Now is the result correct? Does execution speed up? (The answers should now be no and yes.)
3. Put a *critical* directive in front of the update. (Yes and very much no.)
4. Remove the *critical* and add a clause *reduction*(+:quarterpi) to the *for* directive. Now it should be correct and efficient.

Exercise 5 `piadapt`

We continue with exercise 4. We add 'adaptive integration'where needed, the program refines the step size[1]. This means that the iterations no longer take a predictable amount of time.

1. Use the `omp parallel for` construct to parallelize the loop. As in the previous lab, you may at first see an incorrect result. Use the *`reduction`* clause to fix this.
2. Your code should now see a decent speedup, but possible not for all cores. It is possible to get completely linear speedup by adjusting the schedule.

   Start by using *`schedule`*`(static,`*n*`)`. Experiment with values for *n*. When can you get a better speedup? Explain this.
3. Since this code is somewhat dynamic, try *`schedule`*`(dynamic)`. This will actually give a fairly bad result. Why? Use *`schedule`*`(dynamic,$n$)` instead, and experiment with values for *n*.
4. Finally, use *`schedule`*`(guided)`, where OpenMP uses a heuristic. What results does that give?

[1] It doesn't actually do this in a mathematically sophisticated way, so this code is more for the sake of the example.

same exercise

1. Use the `omp parallel for` construct to parallelize the loop. As in the previous lab, you may at first see an incorrect result. Use the `reduction` clause to fix this.
2. Your code should now see a decent speedup, using up to 8 cores. However, it is possible to get completely linear speedup. For this you need to adjust the schedule.
   Start by using `schedule(static,$n$)`. Experiment with values for *n*. When can you get a better speedup? Explain this.
3. Since this code is somewhat dynamic, try `schedule(dynamic)`. This will actually give a fairly bad result. Why? Use `schedule(dynamic,$n$)` instead, and experiment with values for *n*.
4. Finally, use `schedule(guided)`, where OpenMP uses a heuristic. What results does that give?
5. `schedule(auto)` : leave it up to the system.
6. `schedule(runtime)` : leave it up to environment variables; good for experimenting.

Reduction on arrays, static

```
1 // reductarray.c
2 int data[nthreads];
3 #pragma omp parallel for schedule(static,1) \
4   reduction(+:data[:nthreads])
5 for (int it=0; it<nthreads; it++) {
6   for (int i=0; i<nthreads; i++)
7     data[i]++;
8 }
```

Reduction on arrays, dynamic

```
1 int *alloced = (int*)malloc( nthreads*sizeof(int) );
2 for (int i=0; i<nthreads; i++)
3   alloced[i] = 0;
4 #pragma omp parallel for schedule(static,1) \
5   reduction(+:alloced[:nthreads])
6 for (int it=0; it<nthreads; it++) {
7   for (int i=0; i<nthreads; i++)
8         alloced[i]++;
9 }
```

Reductions on C++ vectors

Use the *data* method to extract the array on which to reduce. However, this does not work:

```
1 vector<float> x;
2 #pragma omp parallel reduction(+:x.data())
```

because the reduction clause wants a variable, not an expression, for the array, so you need an extra bare pointer:

```
1 // reductarray.cpp
2 vector<int> data(nthreads,0);
3 int *datadata = data.data();
4 #pragma omp parallel for schedule(static,1) reduction(+:datadata[:nthreads])
```

Reduction on arrays, warning

Each thread gets a copy of the array on the stack
$\Rightarrow$ possible stack overflow

set `OMP_STACKSIZE`

also see `ulimit` on the Unix level.

User-defined reductions

```
1    #pragma omp declare reduction
2    ( identifier : typelist : combiner )
3    [initializer(initializer-expression)]
4
```

where:

`combiner` is an expression that updates the internal variable *omp_out* as function of itself and *omp_in*.

`initializer` sets *omp_priv* to the identity of the reduction;
Often: *initializer*(*omp_priv*=*omp_orig*) to use the initial value.

User-defined reductions, example

```
1 // reductpositive.c
2 int reduce_without_zero(int r,int n);
```

```
1 #pragma omp declare reduction \
2   (rwz:int:omp_out=reduce_without_zero(omp_out,omp_in)) \
3   initializer(omp_priv=-1)
4   m = -1;
5 #pragma omp parallel for reduction(rwz:m)
6   for (int idata=0; idata<ndata; idata++)
7     m = reduce_without_zero(m,data[idata]);
```

Exercise 6

Write a custom reduction that finds the coordinate (x, y) with maximal norm $\sqrt{x^2 + y^2}$. Test on an array of randomly generated coordinates. Decide on how to store this:

```
1 double coords[N][2];
2 double coords[2*N];
3 struct coord { double x,double y };
4 double *coords = (double*) malloc(2*N*sizeof(double));
```

In C++ consider writing a `Coordinate` class and overloading an operator on it.

sample output for prev exercise

```
1 Execute program reductcoord cores 4
2 xy[0] = 0.396465 0.840485
3 xy[1] = 0.353336 0.446583
4 xy[2] = 0.318693 0.886428
5 xy[3] = 0.015583 0.584090
6 xy[4] = 0.159369 0.383716
7 xy[5] = 0.691004 0.058859
8 xy[6] = 0.899854 0.163546
9 xy[7] = 0.159072 0.533065
10 xy[8] = 0.604144 0.582699
11 xy[9] = 0.269971 0.390478
12 Max coordinate: 0.318693 0.886428
```